

COMS30020 - Computer Graphics

Week 2 Briefing

Dr Simon Lock

Where are we ?

Hope that you all got the template project working ?
If not, be sure to do so as quickly as possible !

Aim of this session: to explore this week's workbook
Practical on Friday, but do start the workbook now !

In this workbook we will actually do some drawing !
(That said, it will only be fairly basic stuff - in 2D)

Later we'll be doing more sophisticated stuff in 3D
But we must learn to walk before we can run

Key Technique in Week's Workbook

"Interpolation" is a key tool in Computer Graphics

"Evenly filling-in the gaps between known values"

We will need to apply it to a wide range of entities:

Lines, Colours, Positions, Textures, Shading, Shadows

We first start with simple floating point values:

1.1 ? ? ? 5.5

We then move on to 3-element numerical values:

[1,5,0] [?,?,?] [?,?,?] [?,?,?] [5,1,8]

So we can interpolate RGB colours, 3D positions etc

Interpolation Process

The process of interpolation is always very similar
No matter what type of data we are interpolating
We undertake the same set of activities every time

Let's consider a basic 3 channel RGB interpolation:

ColourInterpolation

Pixel Representation

Typically each colour channel stored in a single byte

These 8 bits allow a value range between 0-255

This may sound familiar from photoshop and HTML !

If we take a 32 bit integer and "pack" it carefully

We can store an RGB colour (including transparency)

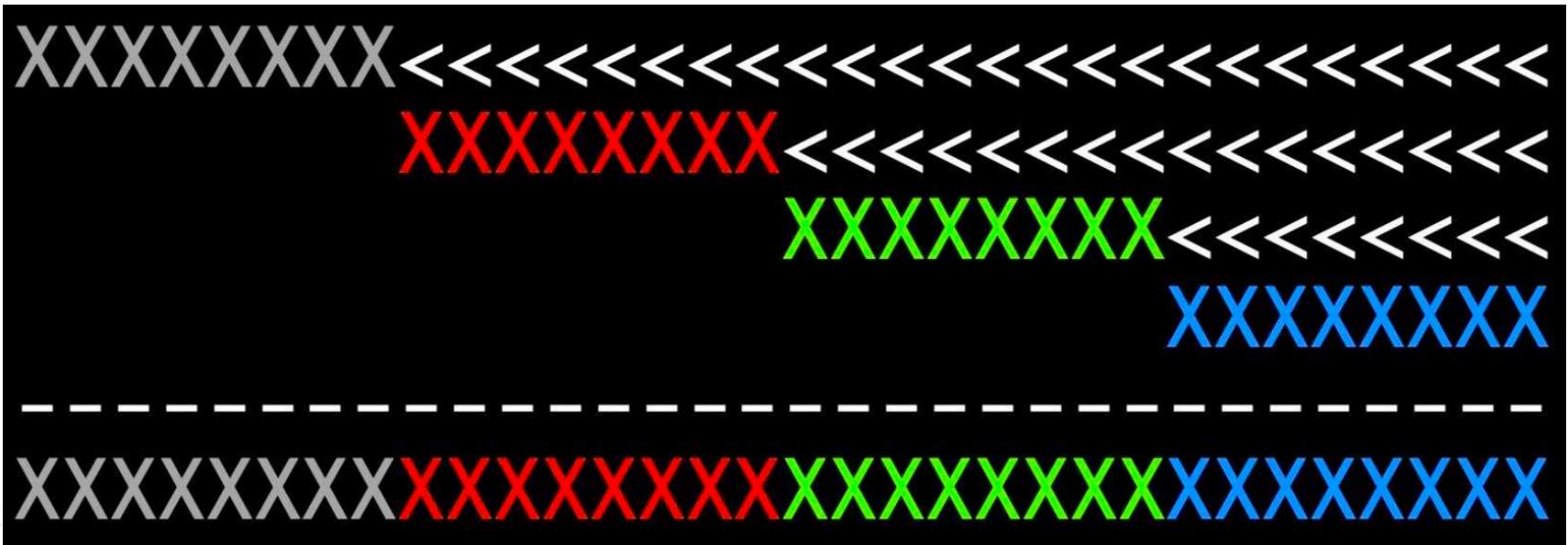
(Being careful to check the colour channel order !)



Packing a Pixel Colour

There are various ways to pack ARGB into an int
Special packing functions, multiplication and so on
We will be using the C/C++ way - "bit shifting":

```
uint32_t colour = (alpha<<24) + (r<<16) + (g<<8) + b;
```



Once our interpolation functions have been written

We can FINALLY do some drawing !!!

Linear Grayscale Interpolation

This is our very first GRAPHICAL task !

Aim is to draw a grayscale linear gradient

Just in one dimension (from left to right)

Repeat on each row (because it's only 1D)



Rectangular Colour Interpolation

Next stage is a 2D colour interpolation

Start with "pure" colours in the four corners

Interpolate DOWN to calculate the first & last columns

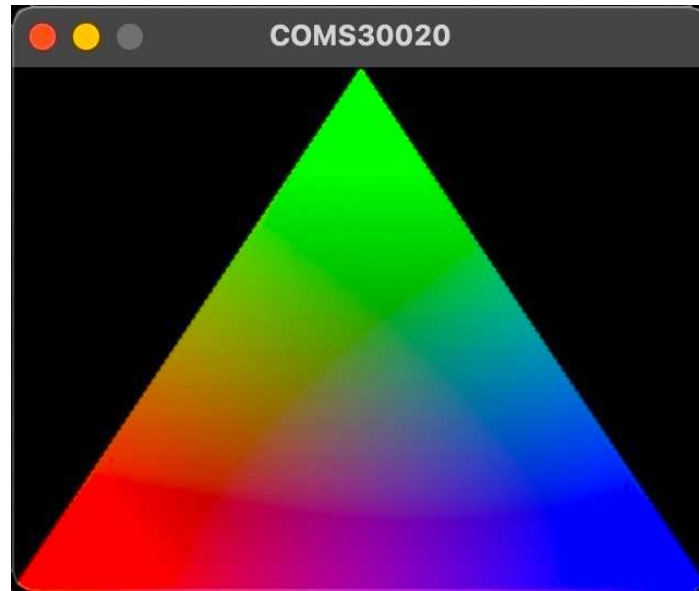
Then interpolate ACROSS to calculate pixels on each row



Triangular Colour Interpolation

If we want primary colour corners, we need a triangle
With a rectangular shape, the sides were perpendicular
A triangle has 3 non-perpendicular sides, so requires...

BarycentricWeighting



Warning !

Your colour spectrum will look a little bit "murky"
Because it is not actually a full colour spectrum

As you move away from a particular vertex...
the value of that primary colour's channel will drop

You'll never get [255,255,0] [255,255,255] [0,0,0]
SUM of ALL channels will be always be exactly 255

This is neither additive nor subtractive colour mixing

What's your Vector Victor ?

This week we'll encounter a slightly annoying issue
We will be using 2 different things called "vector":

- "vector" (from std) used as a dynamic list structure
- "vec3" (from glm) used to store 3 element entities

Need to be careful - think about what you are doing
Ensure you use right structure for a given situation

In reality these are BOTH inappropriate names :o(
We don't get to choose names, so what can you do !

A few words on the use of AI

Prevalence of AI tools

Let's spend a bit of time talking about the use of AI
AI tools to support development are everywhere:

- Web-based chat tools
- Services integrated into code repositories
- Extensions embedded within IDEs

The lure to use AI in development is hard to resist !
(Sometimes we don't even know we are using it !)

Acceptance of AI tools

Rather than trying to ban the use of AI completely
(which would probably be impossible to enforce)

A more pragmatic approach is to permit such tools
For **certain** parts of the development process

In many ways, use of AI support tools is a key skill
Chances are you'll be using them when you graduate

Note that this isn't a "learning how to code" unit
So **careful** use of AI shouldn't conflict with ILOs

Concept Solution

If we could detect code that has been "derived"
We could factor it out during the marking process
(you don't get marks for code that you didn't write)

This would allow us to give credit where credit's due
But allow people to fairly use AI when they need to

This provides flexibility in development process
As well as encouraging people to think critically...
about material produced using generative AI tools

Implementing this Approach

We have a library of 300 solutions to the assignment
ALL submissions of ALL students from previous years

Many are available online (as part of code portfolios)
These will be part of the training set used by AI tools

These 300 solutions also borrow from online examples
(which provides us with an even broader sampling)

All we do is compare YOUR code with this training set
The more similarities that are found...
The more "derivative" your submission has become

The Bottom Line

You **may** use AI tools to SUPPORT development
However, avoid extensive use of AI code generation

AI tools should be there to ASSIST you in development
(such as figuring out obscure compiler error)
It should NOT be doing ALL the work for you !

REMEMBER

Purpose of writing code is to experiment with graphics
You won't gain understanding if all code is generated

Coursework Marking

You will only be rewarded for code that YOU write
Any "derived" code ("found" items or IA generated)
Will be discounted during the marking process
(resulting in a reduced final mark)

Why are we telling you this NOW ?

COURSEWORK UNIT

You will need the codebase you write in weeks 1-8
As a foundation to complete coursework assignment
So the "derivation" rules apply for the entire unit !

EXAMINATION UNIT

If you are taking the exam variant, same rules apply
You need to experiment hands-on with practical tasks
If you just generate your code, you won't learn much



vas 

@vasumanmoza

Follow



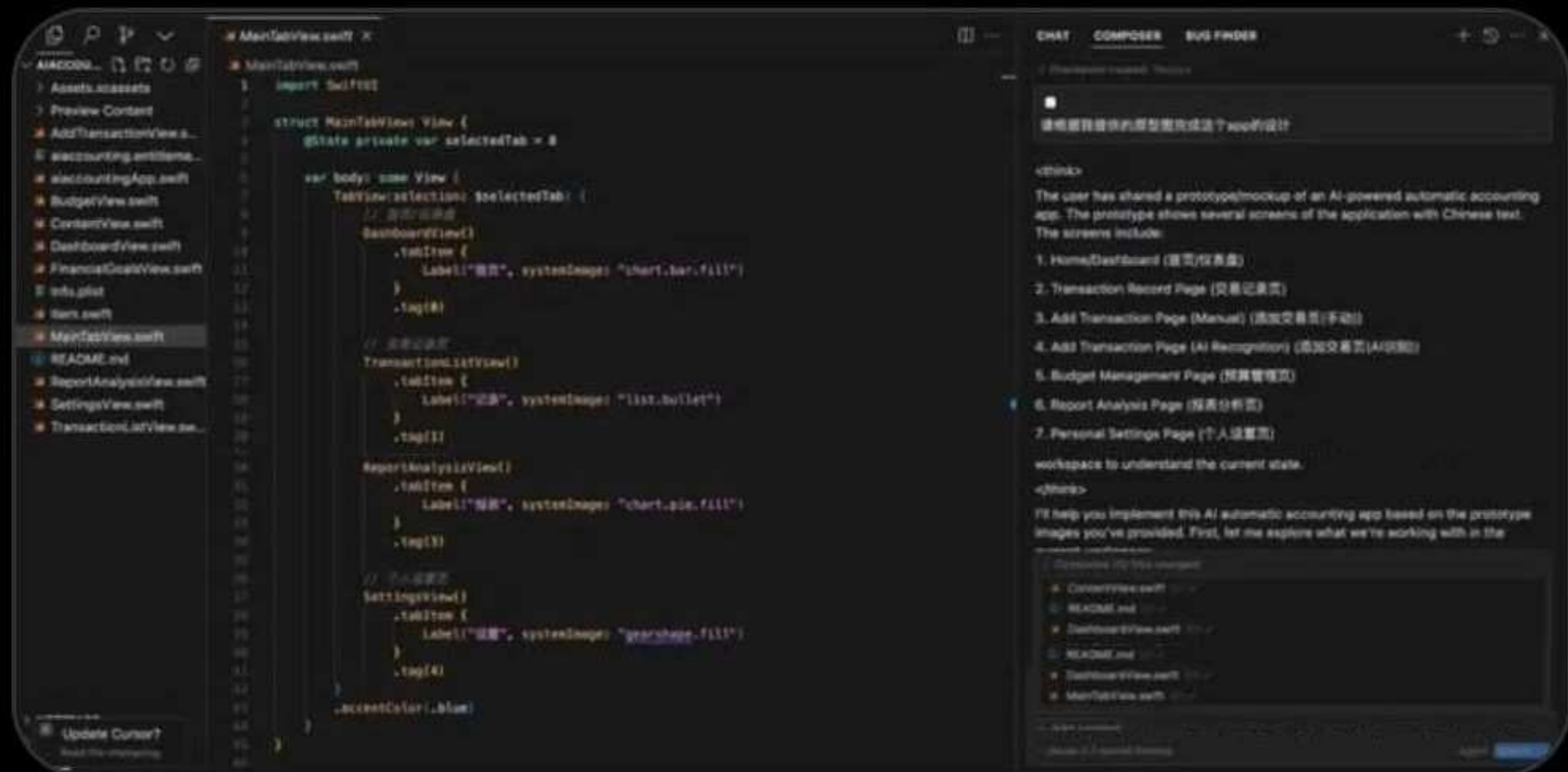
Claude just refactored my entire codebase in one call.

25 tool invocations. 3,000+ new lines. 12 brand new files.

It modularized everything. Broke up monoliths. Cleaned up spaghetti...

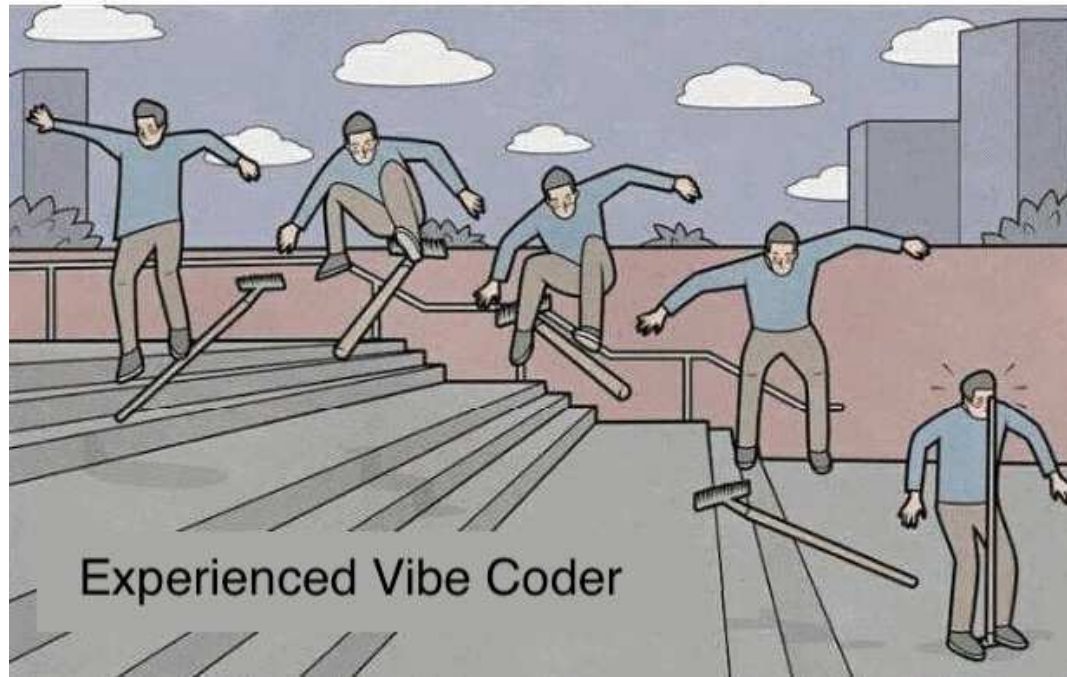
None of it worked.

But boy was it beautiful.





Novice Vibe Coder



Experienced Vibe Coder

Questions ?

I know what you are thinking...

There is BOUND to be some level of similarity
Some standard boilerplate code that everyone has
Or some convergent algorithm that everyone uses

You'd be surprised how much variability there is !

We will be using a variety of filters and thresholds
The aim being to factor out "natural" commonality

Also human-in-the-loop to check for false-positives